Graph Algorithms

Part 3 - CC, SCC, Bipartite Checking, Bipartite Matching

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 10, 2026

Outline

1. **Connectivity and Structure**
   - Connected Components (Undirected)
   - Strongly Connected Components (Directed)

2. **Graph Properties and Matching**
   - Bipartite Checking
   - Bipartite Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connectivity and Structure

Outline

2 Connectivity and Structure

Connected Components (Undirected)

Strongly Connected Components (Directed)

Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

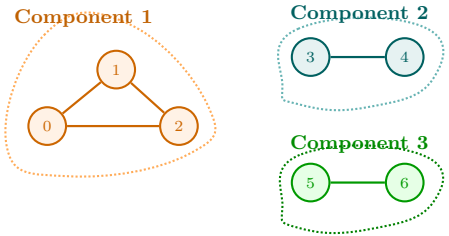
Connected Components

Definition and example

Definition

- A connected component is a maximal set of vertices where every pair of vertices is reachable from one another.
- In an undirected graph, each DFS/BFS from an unvisited vertex returns exactly one component.

Graph with 3 connected components



CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
3 Connected Components (Undirected)
Strongly Connected Components (Directed)
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

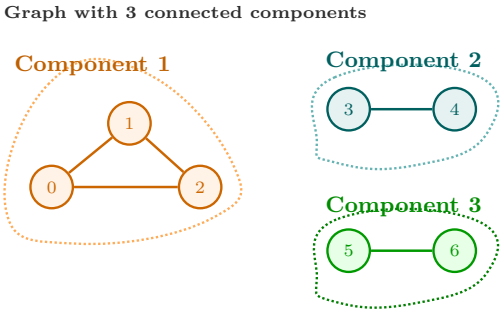
Core idea and basic steps

1. Mark all vertices unvisited.
2. Scan vertices from 0 to $n-1$.
3. If vertex u is unvisited: run DFS/BFS from u, mark all reached vertices as one component.
4. Continue scan until all vertices visited.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Step 1/3 — start from first unvisited vertex



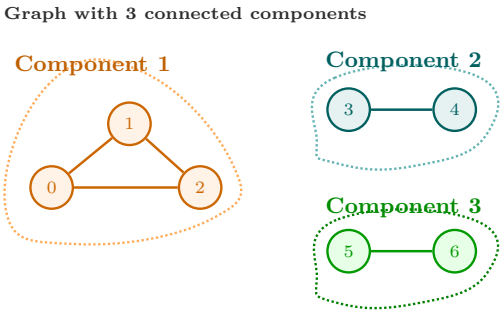
Current step

- Scan from vertex 0.
- Run DFS/BFS from 0.
- Mark all reached vertices as component #1 (orange block).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Step 2/3 — continue scan for next component



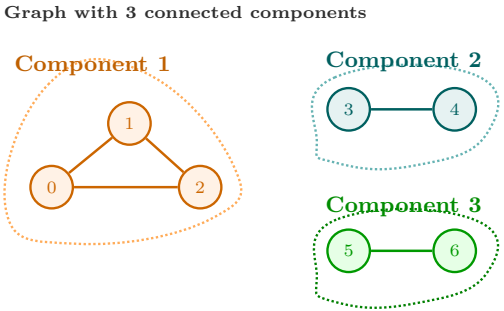
Current step

- Continue scan to next unvisited vertex: 3.
- DFS/BFS from 3 collects component #2 (teal block).
- Vertices in component #1 are skipped as already visited.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Step 3/3 — finish remaining vertices



Current step

- Next unvisited vertex is 5.
- Final DFS/BFS gives component #3 (green block).
- End when all vertices are visited.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Code overview: function contract and key variables

- **Graph model:** `adj` is an **undirected graph** as adjacency list (`adj[u]` = neighbors of vertex `u`), vertices indexed `0..n-1`.
- **Goal:** return both:
  - total number of connected components;
  - component id of each vertex.
- **Main API:** `CCResult connectedComponents(const vector<vector<int>>& adj)`
- **Return type:**
  - `count`: number of components;
  - `compOf[u]`: component id containing vertex `u` (0-based).
- **Important locals:** `n`, `cc`, and DFS helper parameter `cid`.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
8 Connected Components (Undirected)
Strongly Connected Components (Directed)
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Implementation — data type and DFS helper

```cpp
struct CCResult {
    int count;
    vector<int> compOf; // compOf[u] = component id of u
};

void dfsAssign(int u, int cid,
               const vector<vector<int>>& adj,
               vector<int>& compOf) {
    compOf[u] = cid;
    for (int v : adj[u]) {
        if (compOf[v] == -1) dfsAssign(v, cid, adj, compOf);
    }
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Implementation — driver (same routine, continued)

```
CCResult connectedComponents(const vector<vector<int>>& adj) {
    int n = (int)adj.size();
    vector<int> compOf(n, -1);
    int cc = 0;

    for (int u = 0; u < n; ++u) {
        if (compOf[u] != -1) continue; // already assigned
        dfsAssign(u, cc, adj, compOf);
        ++cc;
    }
    return {cc, compOf};
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Complexity by representation

Representation	Complexity
Adjacency list	$O(n+m)$
Adjacency matrix	$O(n^2)$
Edge list only	typically convert to list, then $O(n+m)$

Why these costs?

- With adjacency list, each vertex and each edge is scanned at most once across all DFS/BFS runs $\Rightarrow O(n+m)$.
- With adjacency matrix, checking neighbors of one vertex already costs $O(n)$, so full traversal costs $O(n^2)$.
- Pure edge list is not neighbor-friendly; we usually build adjacency list first, then run DFS/BFS.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
11 Connected Components (Undirected)
Strongly Connected Components (Directed)
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Connected Components

Summary

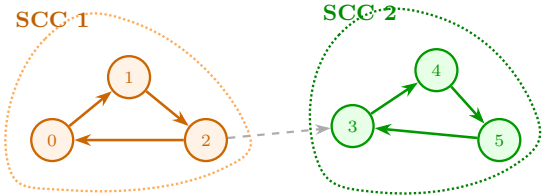
- Best default: adjacency list + DFS/BFS.
- One traversal run = one component.
- This idea is reused in many higher-level graph decompositions.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
12 Connected Components (Undirected)
Strongly Connected Components (Directed)
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Definition and example



Mutually reachable: if $1 \to 2$ is reachable, then $2 \to 1$ is also reachable.

Definition

- SCC: maximal set where every pair of vertices is mutually reachable.
- In directed graphs, SCC is the directed analogue of connected components.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Connected Components (Undirected)

13 Strongly Connected Components (Directed)

Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Core idea and basic steps (Kosaraju)

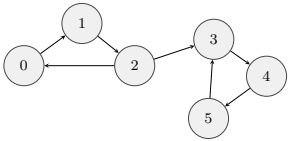
1. DFS on original graph; record finishing order.
2. Reverse all edges.
3. DFS on reversed graph in decreasing finish order.
4. Each DFS tree in step 3 is one SCC.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Step 1/3 — DFS finishing order

Pass 1 on original graph: visited nodes, then record finish order.



- Traverse the **original** graph by DFS.
- When a node finishes, push it to finish-order list.
- Finishing order is the key input for pass 2.

Example finish-order stack (top on right):

```
[5,4,3,2,1,0]
```


CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Connected Components (Undirected)

15 Strongly Connected Components (Directed)

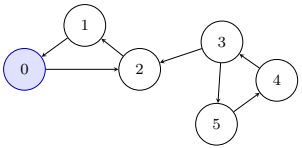
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Step 2/3 — reverse graph and start second pass

Reverse graph G^T: pick next root by finish order (start at 0).



- Build the reversed graph G^T.
- Read vertices in **decreasing** finish order.
- Start DFS from first still-unvisited vertex in this order.

Process vertices in decreasing finish order:

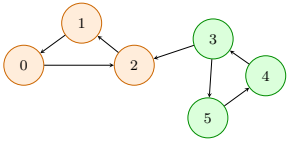
```
0, 1, 2, 3, 4, 5
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Step 3/3 — extract remaining SCCs

DFS on G^T in decreasing finish order extracts SCCs one by one.



$\text{SCC\#1} = \{0,1,2\}, \quad \text{SCC\#2} = \{3,4,5\}$

- One DFS tree in G^T gives exactly one SCC.
- Continue with next unvisited vertex in order.
- Stop when all vertices belong to some SCC.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Connected Components (Undirected)

17 Strongly Connected Components (Directed)

Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Implementation (Kosaraju) — helpers

```
void dfsOrder(int u, const vector<vector<int>>& g, vector<int>& vis,
    vector<int>& order) {
    vis[u] = 1;
    for (int v : g[u]) if (!vis[v]) dfsOrder(v, g, vis, order);
    order.push_back(u); // record finish time
}

void dfsAssign(int u, int cid, const vector<vector<int>>& rg, vector<int>&
    comp) {
    comp[u] = cid;
    for (int v : rg[u]) if (comp[v] == -1) dfsAssign(v, cid, rg, comp);
}
```

- `dfsOrder`: pass 1 on original graph, builds finishing-time order.
- `dfsAssign`: pass 2 on reversed graph, labels one SCC with id `cid`.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Connected Components (Undirected)

18 Strongly Connected Components (Directed)

Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Implementation (Kosaraju) — driver

```
vector<int> kosarajuSCC(const vector<vector<int>>& g) {
    int n = g.size();
    vector<int> vis(n, 0), order, comp(n, -1);
    vector<vector<int>> rg(n);
    for (int u = 0; u < n; ++u) for (int v : g[u]) rg[v].push_back(u);

    for (int u = 0; u < n; ++u) if (!vis[u]) dfsOrder(u, g, vis, order);
    int cid = 0;
    for (int i = n - 1; i >= 0; --i) {
        int u = order[i];
        if (comp[u] == -1) dfsAssign(u, cid++, rg, comp);
    }
    return comp; // comp[u] = SCC id of u
}
```

- Build reversed graph once: `rg`.
- Iterate vertices in decreasing finish order; each new DFS gives one SCC.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
Connected Components (Undirected)
19 Strongly Connected Components (Directed)
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Complexity by representation

Representation	Complexity
Adjacency list (Kosaraju/Tarjan)	$O(n+m)$
Adjacency matrix	$O(n^2)$
Edge list only	build list/reverse list then $O(n+m)$

Why these costs?

- With adjacency list, DFS scans each vertex/edge a constant number of times across both passes $\Rightarrow O(n+m)$.
- With adjacency matrix, exploring neighbors of one vertex is already $O(n)$, so full DFS passes become $O(n^2)$.
- Edge list is not DFS-friendly directly; we usually convert to adjacency lists (original + reversed) first.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
Connected Components (Undirected)
20 Strongly Connected Components (Directed)
Graph Properties and Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Strongly Connected Components

Summary

- SCC compresses a directed graph into a DAG of components.
- Kosaraju: conceptually simple (2 passes).
- Tarjan: one-pass alternative with low-link values.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
Connected Components (Undirected)
21 Strongly Connected Components (Directed)
Graph Properties and Matching

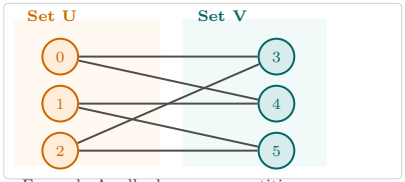
ltsach@hcmut.edu.vn
Thanh-Sach LE

Graph Properties and Matching

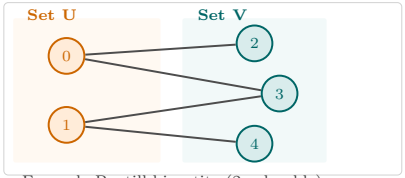
ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking

Definition and example



Example A: all edges cross partitions.



Example B: still bipartite (2-colorable).

Definition

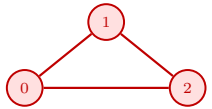
- A graph is bipartite iff vertices can be split into 2 sets and every edge crosses sets.
- Equivalent condition: no odd-length cycle.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking

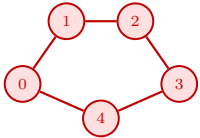
Definition and example (cont.)

Example C: odd cycle (3-cycle)



Not bipartite: odd cycle forces color conflict.

Example D: odd cycle (5-cycle)



Not bipartite: odd-length cycle present.

Counterexample insight

- Odd cycles (length 3, 5, ...) are not 2-colorable.
- So any graph containing an odd cycle is not bipartite.

Key equivalence: Bipartite $\Leftrightarrow$ 2-colorable $\Leftrightarrow$ no odd cycle.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Graph Properties and Matching

24 Bipartite Checking

Bipartite Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking

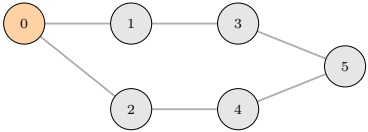
Core idea and basic steps

1. Initialize all vertices uncolored.
2. For each uncolored vertex, BFS/DFS and color alternately.
3. If an edge connects two same-color vertices $\Rightarrow$ not bipartite.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Pass case)

Step 0 — initialize BFS from vertex 0



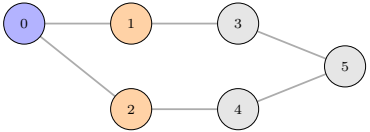


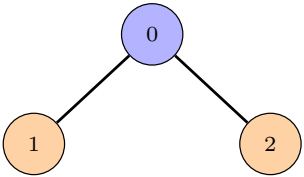
- Start BFS at vertex 0.
- Color(0)=Left set.
- Queue: [0]

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Pass case)

Step 1 — expand 0, discover 1 and 2



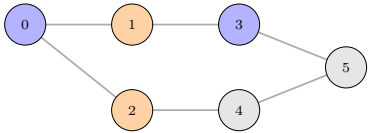


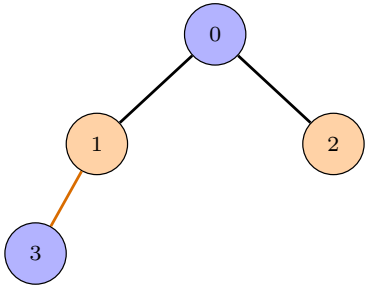
- Visit neighbors of 0.
- Assign opposite color to 1,2.
- Queue: [1, 2]

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Pass case)

Step 2 — expand 1, discover 3



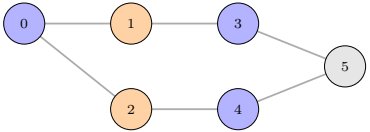


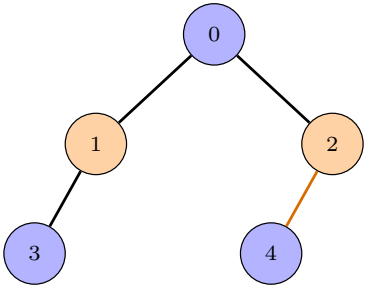
- Pop 1, inspect edges (1,0), (1,3).
- 3 is uncolored ⇒ color opposite to 1.
- Queue: [2, 3]

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Pass case)

Step 3 — expand 2, discover 4



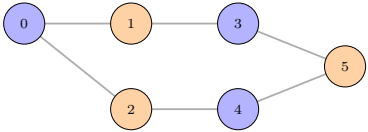


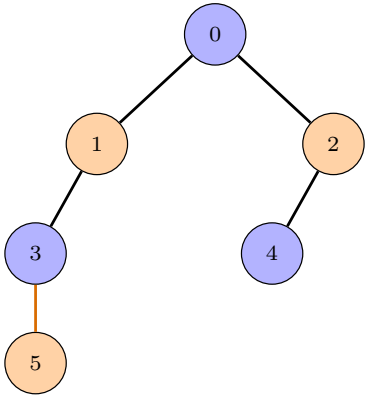
- Pop 2, inspect edges (2,0), (2,4).
- 4 is uncolored ⇒ color opposite to 2.
- Queue: [3, 4]

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Pass case)

Step 4 — expand 3, discover 5



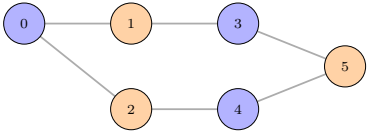


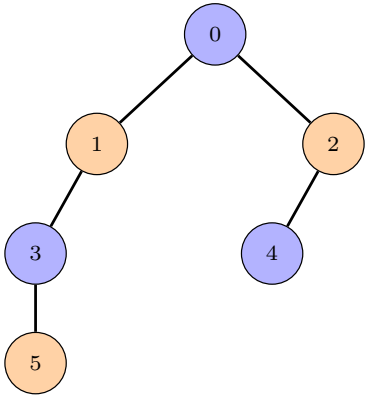
- Pop 3, inspect edges (3,1), (3,5).
- 5 is uncolored ⇒ color opposite to 3.
- Queue: [4, 5]

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Pass case)

Step 5 — finish traversal, no conflict

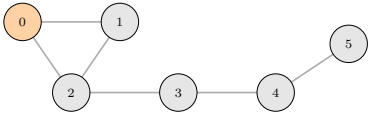




- Pop 4 then 5: all neighbor checks are consistent.
- No edge with same-color endpoints.
- Result: graph is bipartite.

Bipartite Checking (Fail case)

Step 0 — initialize on a 6-node non-bipartite graph



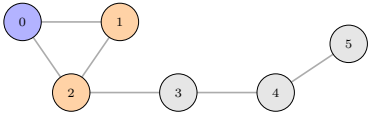


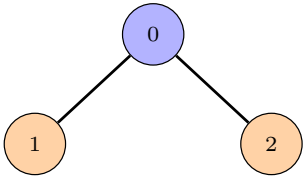
- Start BFS at 0.
- This graph has a hidden odd cycle $(0, 1, 2, 0)$.
- Queue: [0]

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Fail case)

Step 1 — expand 0, color 1 and 2



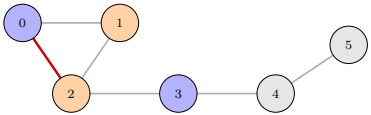


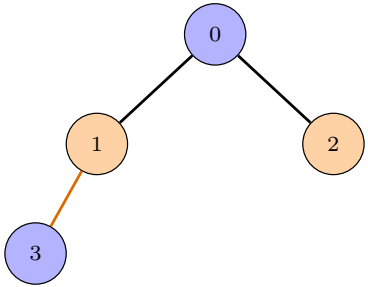
- Color neighbors of 0 with opposite color.
- Queue: [1, 2]
- No conflict yet.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Fail case)

Step 2 — expand 1, discover 3



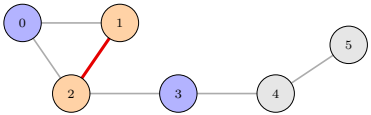


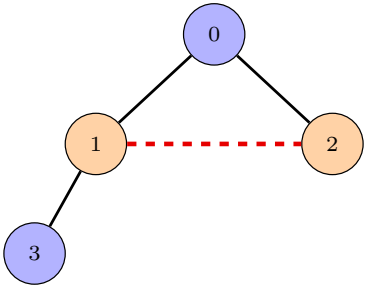
- Pop 1: edge (1,3) gives Color(3)=blue.
- Queue: [2, 3]
- Next check of edge (1,2) is problematic.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking (Fail case)

Step 3 — conflict found on edge (1,2)





- Edge (1,2) connects two vertices with same color (orange).
- BFS stops immediately and returns `false`.
- Conclusion: graph is **not bipartite**.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking

Implementation (BFS coloring)

```
bool isBipartite(const vector<vector<int>>& adj) {
    int n = adj.size(); vector<int> color(n, -1);
    queue<int> q;
    for (int s = 0; s < n; ++s) if (color[s] == -1) {
        color[s] = 0; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) color[v] = color[u] ^ 1, q.push(v);
                else if (color[v] == color[u]) return false;
            }
        }
    }
    return true;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking

Complexity by representation

Representation	Complexity
Adjacency list	$O(n+m)$
Adjacency matrix	$O(n^2)$
Edge list only	usually build list then $O(n+m)$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Checking

Summary

- Fast structural test with BFS/DFS coloring.
- Foundation for many matching and scheduling models.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Definition and example

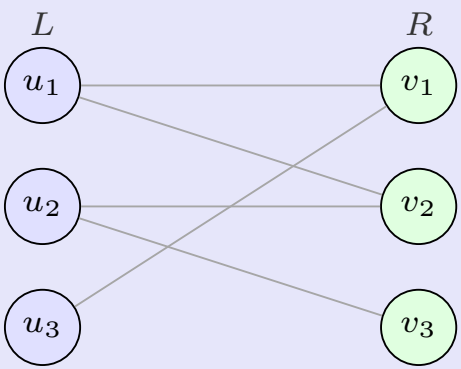
- **Bipartite graph:** vertices are split into two disjoint sets L and R, and every edge goes from L to R.
- **Matching:** a set of edges where no two edges share a vertex.
- **Bipartite matching problem:** find a matching with maximum size.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Example: input graph and expected output

Input graph



$E = \{(u_1, v_1), (u_1, v_2), (u_2, v_2), (u_2, v_3), (u_3, v_1)\}$

Expected output

Maximum matching size is

$$|M| = 3.$$

One valid optimal matching:

$$M = \{(u_1, v_2), (u_2, v_3), (u_3, v_1)\}.$$

- Every vertex is used at most once.
- Here all 3 vertices in L are matched.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Core idea and basic steps



1. Start from current matching (possibly empty).
2. Find an **augmenting path** between unmatched vertices.
3. Flip matched/unmatched status along that path.
4. Repeat until no **augmenting path** exists.

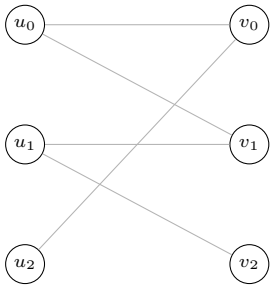
Definition: an **augmenting path** is an alternating path (unmatched, matched, unmatched, ...) that starts and ends at unmatched vertices.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 1 — setup and loop order

Graph view (U-V)



$U = \{u_0, u_1, u_2\}$

$V = \{v_0, v_1, v_2\}$

State + DFS view

seen[u]

0	1	2	0/1 flags

mt[v]

$v_0 \to -$ $v_1 \to -$ $v_2 \to -$

Current call stack

for(u=0..2), start with `u=0`

DFS tree

not started

Insight

We follow the exact loop order in code.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Graph Properties and Matching

Bipartite Checking

42 Bipartite Matching

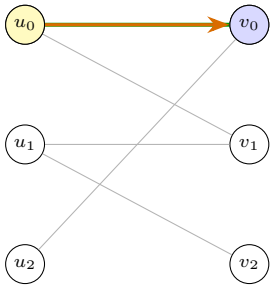
ltsach@hcmut.edu.vn

Thanh-Sach LE

Bipartite Matching

Step 2 — iterate u=u_0

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

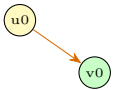
mt[v]

$v_0 \to u_0 \quad v_1 \to - \quad v_2 \to -$

Current call stack

`aug(u_0)`

DFS tree



v0 is free ⇒ match immediately.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Graph Properties and Matching

Bipartite Checking

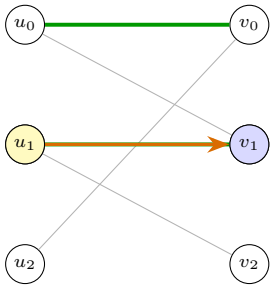
43 Bipartite Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 3 — iterate u=u_1

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

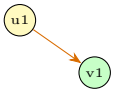
mt[v]

$v_0 \to u_0$ $v_1 \to u_1$ $v_2 \to -$

Current call stack

`aug(u_1)`

DFS tree



Second direct match: size grows to 2.

CC, SCC, Bipartite Checking, Bipartite Matching

Outline

Connectivity and Structure

Graph Properties and Matching

Bipartite Checking

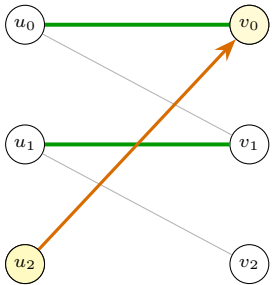
44 Bipartite Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 4 — iterate u=u_2: start DFS

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

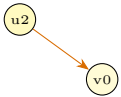
mt[v]

$v_0 \to u_0$ $v_1 \to u_1$ $v_2 \to -$

Current call stack

aug(u_2)

DFS tree



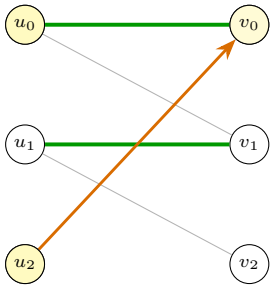
v0 is occupied by u0 ⇒ recurse.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 5 — recurse from v_0 to aug(u_0)

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

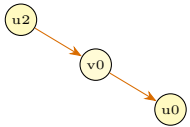
mt[v]

$v_0 \to u_0$ $v_1 \to u_1$ $v_2 \to -$

Current call stack

`aug(u_2) -> aug(u_0)`

DFS tree



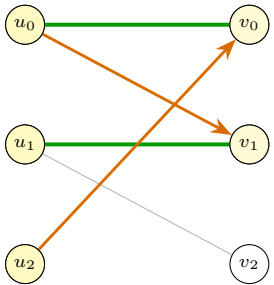
v_0 is matched with u_0, recurse to `aug(u_0)`.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 6 — recurse from v_1 to aug(u_1)

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

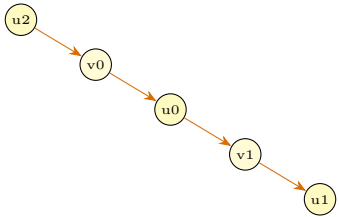
mt[v]

$v_0 \to u_0$ $v_1 \to u_1$ $v_2 \to -$

Current call stack

`aug(u_2) -> aug(u_0) -> aug(u_1)`

DFS tree



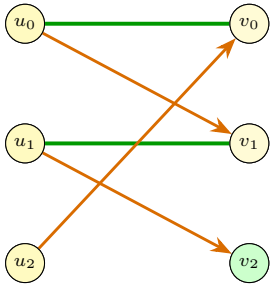
v_1 is matched with u_1, recurse to `aug(u_1)`.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 7 — hit free node v_2 (success leaf)

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

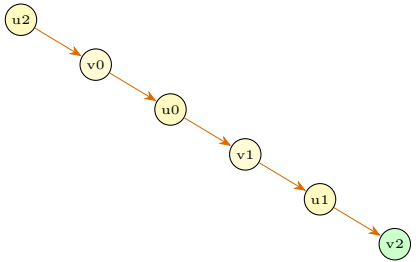
mt[v]

$v_0 \to u_0$ $v_1 \to u_1$ $v_2 \to -$

Current call stack

```
aug(u_2) -> aug(u_0) -> aug(u_1)
```

DFS tree

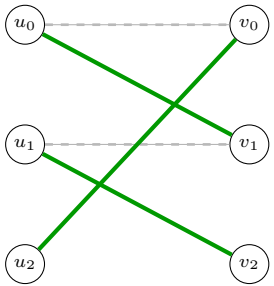


ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 8 — unwind and flip along path

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

mt[v]

$v_0 \to u_2$ $v_1 \to u_0$ $v_2 \to u_1$

Alternating path flip

before: u2-v0==u0-v1==u1-v2 (v2 free end)

after: old matched edges become free, new path edges

Insight

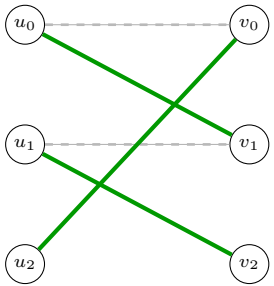
This is the core Kuhn idea: recursive reassignment.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Step 9 — finish aug(u_2) and increment ans

Graph view (U-V)



State + DFS view

seen[u]

0	1	2

0/1 flags

mt[v]

$v_0 \to u_2$ $v_1 \to u_0$ $v_2 \to u_1$

Alternating path flip

before: u2-v0==u0-v1==u1-v2 (v2 free end)

after: old matched edges become free, new path edges

Insight

This is the core Kuhn idea: recursive reassignment.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Implementation (Kuhn DFS)

```
bool aug(int u, const vector<vector<int>>& g, vector<int>& seen,
     vector<int>& mt) {
    if (seen[u]) return false; seen[u] = 1;
    for (int v : g[u]) if (mt[v] == -1 || aug(mt[v], g, seen, mt)) {
        mt[v] = u; return true;
    }
    return false;
}
int kuhnMaxMatching(int nL, int nR, const vector<vector<int>>& g) {
    vector<int> mt(nR, -1); int ans = 0;
    for (int u = 0; u < nL; ++u) {
        vector<int> seen(nL, 0);
        if (aug(u, g, seen, mt)) ++ans;
    }
    return ans;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Complexity by representation

Algorithm / representation	Complexity
Kuhn + adjacency list	$O(nm)$
Hopcroft–Karp + adjacency list	$O(m\sqrt{n})$
Adjacency matrix variants	usually slower; sparse lists preferred

ltsach@hcmut.edu.vn
Thanh-Sach LE

Bipartite Matching

Summary

- Matching solves assignment-like constraints.
- Augmenting path is the key invariant.
- Hopcroft–Karp is the standard fast choice.

CC, SCC, Bipartite Checking, Bipartite Matching
Outline
Connectivity and Structure
Graph Properties and Matching
Bipartite Checking
53 Bipartite Matching

ltsach@hcmut.edu.vn
Thanh-Sach LE